ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Report

on

Mental Health Case Tracking System

by

Ashish SK Gupta
150096725084

Index

1. Introduction to the Case Study

Mental health is one of the most sensitive areas of healthcare. Patients require long-term, confidential, and well-organised tracking of their treatment history. This case study presents a Mental Health Case Tracking System built entirely in C++ using only the standard library.

The system is designed at beginner-level CSE student difficulty — no external libraries, no database, and no complex frameworks. It demonstrates how fundamental C++ concepts such as file I/O, string parsing, functions, and control flow can be combined to build a practical, secure application.

2. Problem Statement / Case Background (Abstract)

Background

Mental health counsellors currently manage patient records on paper or in unencrypted files. This creates risks of data breaches, loss of records, and inability to efficiently track long-term treatment. A purpose-built digital system with access control is needed.

Abstract

This case study presents the design and implementation of a Mental Health Case Tracking System using C++. The system provides secure login and registration with FNV-1a password hashing, unique case registration with severity tracking, session logging per case, and pipe-delimited flat-file storage (.dat files). The entire solution is implemented in under 200 lines of beginner-friendly C++ using only standard library headers.

3. Case Study Design

The system is designed as a modular pipeline with the following stages:

8. Authentication Module — Users register or login. Passwords are hashed using FNV-1a before storage. A default admin account is seeded on first run.
9. Case Management Module — Cases are registered with auto-generated unique IDs (C0001, C0002...), patient initials, age, primary concern, severity, timestamp, and status.
10. Session Tracking Module — Therapy sessions are logged with auto-generated session IDs (S00001...), linked to a case ID, with date, duration in minutes, and brief notes.
11. Storage Module — All data is persisted across runs in pipe-delimited .dat files (users.dat, cases.dat, sessions.dat). No external database is needed.
12. CLI Menu / Dashboard — A clean text menu ties all modules together for post-login navigation.

4. Methods & Algorithms / Technology Applied

Key Methods

- FNV-1a 64-bit hash for password hashing — fast, portable, no external library needed.
- Pipe-delimited flat-file storage — fields separated by | to safely handle spaces in text.

- Auto-incrementing ID generation — formatted with setw/setfill for zero-padded IDs.
- String splitting via stringstream — splits pipe-delimited lines into fields for reading records.
- In-place file rewriting — for status updates, all lines are read into a vector, modified, then rewritten.

Technology Stack

- Language: C++ (C++11 or later)
- Headers: iostream, fstream, sstream, string, vector, iomanip, ctime, algorithm
- Storage: .dat flat files (pipe-delimited text)
- Build: g++ mental_health_tracker.cpp -o tracker
- Environment: Any terminal — Windows, Linux, or macOS

5. Implementation Details

The code is organized in a single file (mental_health_tracker.cpp) with clearly separated functions:

- hashPwd(pwd) — FNV-1a 64-bit hash, returns a 16-char hex string.
- now() — returns current date/time formatted as YYYY-MM-DD HH:MM using ctime.
- readFile() / writeFile() / appendLine() — generic file helpers for reading/writing .dat records.
- split(s) — splits a pipe-delimited string using stringstream, returns a vector of tokens.
- seedAdmin() — creates default admin account on first run if users.dat does not exist.
- doRegister() / doLogin() — handle registration and login with password hashing and verification.
- addCase() — prompts for patient details, assigns a zero-padded case ID, appends to cases.dat.
- showCases() — reads and displays all cases in a formatted table.
- updateStatus() — reads all cases into a vector, modifies the target row, rewrites the file.
- addSession() — validates case exists, logs session details to sessions.dat with auto-generated ID.
- viewSessions() — displays all sessions, optionally filtered by case ID.
- dashboard() / main() — CLI navigation loops using while and if-else chains.

6. Results and Conclusion

Results

- Confidential case management achieved through secure login and hashed passwords.
- Case registration works with unique IDs, severity levels, and timestamps.
- Session tracking successfully links sessions to cases with notes and duration.

- All data persists across program restarts using .dat flat files.
- System compiles and runs with zero errors using standard g++ on any platform.
- Entire implementation is under 200 lines of clean, readable C++ code.

Conclusion

This case study demonstrates a complete Mental Health Case Tracking System in C++ that addresses all stated objectives: confidential case management, session tracking, secure login with password hashing, and persistent storage. The implementation intentionally uses only C++ standard library features, making it fully accessible to beginner-level CSE students while demonstrating good software engineering practices such as modular design, secure authentication, and persistent data management.

7. References

13. C++ Standard Library Reference — https://en.cppreference.com/
14. C++ fstream documentation — https://cplusplus.com/reference/fstream/
15. FNV Hash Algorithm — http://www.isthe.com/chongo/tech/comp/fnv/
16. ITM Skills University, School of Future Tech — Case Study 31 Problem Statement
17. C++ iomanip reference (setw, setfill) — https://cplusplus.com/reference/iomanip/